MCP :

- Follows a Client-Server Architecture.
    - MCP is fundamentally divided into Two Layers.
    - One is the Transport Layer. The other one is the Data Layer.
    - **Conceptually** the **Data layer** is the inner layer, while the **Transport layer** is the outer layer.
    - MCP is a bidirectional bridge, both sides are capable of exposing specific features to the other.

Transports Layer :

- **Transport Layer** defines *how* the package physically moves between the client and the server.
- Defines the communication mechanisms and channels that enable data exchange between clients and servers.
- Including transport-specific connection establishment, message framing, and authorisation.

MCP (Host):

- The AI application that coordinates and manages one or multiple MCP clients
    - **MCP Host** an AI Application (Claude Code / Claude Desktop) Establish Connection to one or more MCP Servers.
    - **MCP Host** Accomplishes This: By Creating **MCP Client** for each an every **MCP Server**.

MCP (Client):

- A component that maintains a connection to an MCP server and obtains context from an MCP server for the **MCP Host** to use
    - Each **MCP Client** maintains a dedicated connection with its corresponding **MCP Server**.

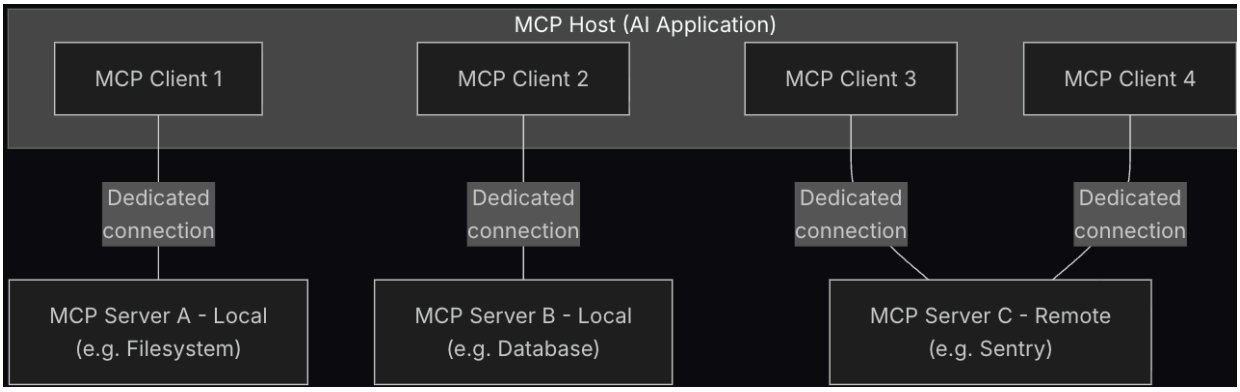
MCP (Server):

STDIO (Standard Input/Output) Transport Model

- Foundational connection mechanism used in the MCP.
    - 1-to-1 (Isolated & Dedicated)
    - So it Serve a single **MCP Client**
    - **Dedicated Resources =>** If you open three different client applications that all use the same STDIO tool, the system will actually spin up **three separate instances** of that server process.
    - **No Shared State =>** Each process runs in its own memory sandbox. Server A has no idea that Server B or C exists, even though they are running the exact same code.
    - **Fate Sharing =>** The moment you close the client application, the operating system automatically terminates the child process. They live and die together.
    - **Examples:** Local development, desktop apps, and system

automations
    - Choose **STDIO =>** Maximum Security, Absolute Zero Configuration for the end User, isolated playground where the client has total ownership over the tool lifecycle.
    -

The Network / Remote Model (SSE/HTTP)

- Remote MCP Servers that uses streamable HTTP Transport will Typically Serve Many MCP Clients.
    - Single Instance, Multiple Connections => The server is started once (Background Demon or Cloud Service) and listens on a specific port or URL.
    - Shared Environment => Dozens of different clients, users, or remote AI agents can send HTTP requests to that single running process simultaneously.
    - State Persistence => Because it runs independently of the clients, the server stays alive even if all clients disconnect. It can easily maintain a single, shared database connection or cache across all users.
    - **Examples:** Web deployments, production architectures, and remote nodes
    - Choose Network (Remote SSE / HTTP) **=>** Centralised Service, Share Heavy Resources (Massive Database Pool or An Active AI Model) across multiple users, Host the tool on a Completely Different Machine
- 

Data Layer :

- **Data Layer** defines *what* is inside the package (the rules, primitives, and content)
- Defines The **JSON-RPC 2.0** Protocol for client-server communication.
- It handles how an AI model interacts with tools and data, using standard **JSON-RPC 2.0** as its messaging format.
- **JSON-RPC 2.0** which is light weight, text-based formate for sending commands and data back and forth.

Discovery :

- Before Client and Server do anything else, they need to introduce themselves. This thing called initial **Hand-Shake**.
  - Through server/discovery request client ask:
    - What version of the protocol do you speak?
    - What are your capabilities?
    - Who exactly are you?

🤖Server Feature :

- This allows the Server to tell the Client. What it can actually **do or Provide**.
  - It breaks down into three offering
    - **Tools :** Actionable tasks the AI can execute (e.g., "Run this Python code" or "Fetch this web page").
    - **Resources :** Contextual data the AI can read (e.g., "Here is the content of file_X.txt" or "Here is the latest error log").
    - **Prompts :** Pre-made templates or shortcuts for interacting with the user (e.g., "Use this specific template to ask the user for their API key").

👤 Client Features:

- This gives the server a way to ask the user or the client application for information.
- Enables servers to **elicit (Richer)** input from the user.
- *Note on "Sampling is deprecated":* Previously, servers could ask the AI client to generate (sample) a response or a piece of text. As of the July 2026 update, this specific feature is being phased out and shouldn't be used anymore.

🛠️ Utility Features:

- These are background helper functions that make the user experience smoother:
  - **Notifications :** Real-time, one-way alerts (e.g., "The database just disconnected" or "Data updated").
  - **Progress Tracking :** A loading bar or status update for tasks that take a long time to finish (e.g., "Downloading file... 45% complete").
  - **Configuration**
  - **Cancellation**
  - **Error Reporting**

Data Layer Protocol :

- A core part of MCP is defining the schema and semantics between MCP clients and MCP servers.
- Developers will likely find the data layer — in particular.
- Developers care most about the **primitives** (the fundamental building blocks like Tools, Resources, and Prompts) because that is how they actually make their data accessible to an AI.

Primitives :

- MCP primitives are the most important concept within MCP.
- They define what clients and servers can offer each other.
- Each primitive type has associated methods.
- For **Discovery: */list**, **Retrieval: */get**, **Execution: tool/call**
- **MCP Clients** will use the ***/list** methods to discover available primitives.
- **Example:** A client can first list all available tools (**tools/list**) and then execute them. This design allows listings to be dynamic
- These primitives specify the types of contextual information that can be shared with AI applications and the range of actions that can be performed.
- MCP defines three core primitives that *servers* can expose:

    - **Tools**:
        - Executable functions that AI applications can invoke to perform actions (e.g., file operations, API calls, database queries)
        - How you expose actions (like running a database query or sending an email) so the AI can execute them.

    - **Resources**:
        - How you expose data (like a database schema, a file, or API documentation) so the AI can read it.
        - Data sources that provide contextual information to AI applications (e.g., file contents, database records, API responses).

    - **Prompts**:
        - Reusable templates that help structure interactions with language models (e.g., system prompts, few-shot examples).
        - How you expose pre-defined templates so the AI knows how to structure its questions or interactions.

MCP Server Side (Primitive):

- These are the capabilities that a MCP Server offers to a MCP Client

(AI Application / MCP Client).
- They Allow LLM to Access Data and take Actions.
  - **Resources:**
    - Passive data sources that provide read-only access to information for context, such as file contents, database schemas, or API documentation.
    - That provides context about the DataBase by the Schema of the Database.

  - **Tools:**
    - Functions that your LLM can actively call, and decides when to use them based on user requests.
    - Tools can write to databases, call external APIs, modify files, or trigger other logic
    - It can expose tools for querying the database.

  - **Prompt:**
    - Pre-built instruction templates that tell the model to work with specific tools and resources.
    - That includes a few short examples for interacting with tools.

- **Examples:**
  - File system Servers for document access
  - Data Base Servers For Data Query.
  - Github Server for Code Management.
  - Slack Servers for Team Communications.
  - Calendar Servers for Scheduling.

- So I am right that the **Claude Desktop App**, those "**Connectors**" or **Plugins** you configure are **MCP Servers**.

TOOLS :
- Tools enable AI models to perform actions.
- Each tool defines a specific operation with typed inputs and outputs.
- The model requests tool execution based on context.
  - **How Tools Work**
    - Tools are schema-defined interfaces that LLMs can invoke.
    - MCP uses JSON Schema for validation.
    - Each tool performs a single operation with clearly defined inputs and outputs.
    - Tools may require user consent prior to execution, helping to ensure users maintain control over actions taken by a model.

      Protocol operations:
    - **tools/list** => Discover available tools => Array of tool

definitions with schemas
- **tools/call** => Execute a specific tool => Tool execution result

- **Example tool definition:**

```
{
  name: "searchFlights",
  description: "Search for available flights",
  inputSchema: {
    type: "object",
    properties: {
      origin: { type: "string", description: "Departure
city" },
      destination: { type: "string", description: "Arrival
city" },
      date: { type: "string", format: "date", description:
"Travel date" }
    },
    required: ["origin", "destination", "date"]
  }
}
```

```
Flight Search
searchFlights(origin: "NYC", destination: "Barcelona", date:
"2024-06-15")

Calendar Blocking
createCalendarEvent(title: "Barcelona Trip", startDate:
"2024-06-15", endDate: "2024-06-22")

Email notification
sendEmail(to: "team@work.com", subject: "Out of Office",
body: "...")
```

Resources :

- Resources provide structure access to the information that AI Application can retrieve provide to model as context.

How Resources Work :

- Resources expose data from files, APIs, databases, or any other source that an AI needs to understand context.

Resources support two discovery patterns:

- **Direct Resources** - fixed URIs that point to specific data. Example: calendar://events/2024 - returns calendar availability for 2024
- **Resource Templates** - dynamic URIs with parameters for flexible queries.

- Example:
  - travel://activities/{city}/{category} => returns activities by city and category
  - travel://activities/barcelona/museums => returns all museums in Barcelona

- **Protocol operations:**

Method	Purpose	Returns
`resources/list`	List available direct resources	Array of resource descriptors
`resources/templates/list`	Discover resource templates	Array of resource template definitions
`resources/read`	Retrieve resource contents	Resource data with metadata
`subscriptions/listen`	Monitor resource changes	Stream of update notifications

  - To watch specific resources for changes, a client sends a subscriptions/listen request with the resource URIs listed in the resourceSubscriptions filter. The server delivers **notifications/resources/updated** on the resulting stream whenever a watched resource changes.

Example: Getting Travel Planning Context :
- **Calendar data** (calendar://events/2026) - Checks user availability.
- **Travel documents** (file:///Documents/Travel/passport.pdf) - Accesses important documents.
- **Previous itineraries** (trips://history/UnitedKingdom-2024) - References past trips and preferences.

The AI application retrieves these resources and decides how to process them, whether selecting a subset of data using embeddings or keyword search, or passing raw data directly to the model.

Resource Template Examples:

```
{
  "uriTemplate": "weather://forecast/{city}/{date}",
  "name": "weather-forecast",
  "title": "Weather Forecast",
  "description": "Get weather forecast for any city and
```

```
date",
  "mimeType": "application/json"
}

{
  "uriTemplate": "travel://flights/{origin}/{destination}",
  "name": "flight-search",
  "title": "Flight Search",
  "description": "Search available flights between cities",
  "mimeType": "application/json"
}
```

Parameter Completion :
Dynamic resources support parameter completion. For example:

- Typing “Par” as input for weather://forecast/{city} might suggest “Paris” or “Park City”
- Typing “JFK” for flights://search/{airport} might suggest “JFK - John F. Kennedy International”

The system helps discover valid values without requiring exact format knowledge.

Prompts :

- Prompts are structured templates that define expected inputs and interaction patterns.
- They are user-controlled, requiring explicit invocation rather than automatic triggering.
- Prompts can be context-aware, referencing available resources and tools to create comprehensive workflows.
- Similar to resources, prompts support parameter completion to help users discover valid argument values.

Protocol operations:

Method	Purpose	Returns
`prompts/list`	Discover available prompts	Array of prompt descriptors
`prompts/get`	Retrieve prompt details	Full prompt definition with arguments

“Plan a vacation” prompt:

```
{
  "name": "plan-vacation",
  "title": "Plan a vacation",
  "description": "Guide through vacation planning process",
```

```
  "arguments": [
    { "name": "destination", "type": "string", "required":
true },
    { "name": "duration", "type": "number", "description":
"days" },
    { "name": "budget", "type": "number", "required":
false },
    { "name": "interests", "type": "array", "items":
{ "type": "string" } }
  ]
}
```

Example: Multi-Server Travel Planning :

- **Travel Server** - Handles flights, hotels, and itineraries
- **Weather Server** - Provides climate data and forecasts
- **Calendar/Email Server** - Manages schedules and communications

The Complete Flow :

- **User invokes a prompt with parameters:**

```
{
  "prompt": "plan-vacation",
  "arguments": {
    "destination": "United Kingdom",
    "departure_date": "2024-06-15",
    "return_date": "2024-06-22",
    "budget": 3000,
    "travelers": 2
  }
}
```

- **User selects resources to include:**
  - calendar://my-calendar/June-2024 (from Calendar Server)
  - travel://preferences/europe (from Travel Server)
  - travel://past-trips/Spain-2023 (from Travel Server)

- **AI processes the request using tools:**
  - The AI first reads all selected resources to gather context - identifying available dates from the calendar, learning preferred airlines and hotel types from travel preferences, and discovering previously enjoyed locations from past trips.
  - Using this context, the AI then executes the prompt provided by the AI application.
  - In our example, the AI application exposes the weather tools from the connected MCP weather server to the model. Because weather can affect travel plans, the AI chooses to call checkWeather()

when interpreting the prompt.
As a result the AI executes a series of tools:
- searchFlights() - Queries airlines for NYC to Barcelona flights
- checkWeather() - Retrieves climate forecasts for travel dates

– The AI then uses this information to create the booking and following steps, requesting approval from the user where necessary:
- bookHotel() - Finds hotels within the specified budget
- createCalendarEvent() - Adds the trip to the user's calendar
- sendEmail() - Sends confirmation with trip details

MCP Client Side (Primitive):

- MCP clients are instantiated by host applications to communicate with particular MCP servers.
- Each client handles one direct communication with one server.

Core Client Features :

- In addition to making use of context provided by **MCP Servers**, **MCP Clients** may provide several features to **MCP Servers**.

Feature	Explanation	Example
Elicitation	Elicitation enables servers to request specific information from users during interactions, providing a structured way for servers to gather information on demand.	A server booking travel may ask for the user's preference on airplane seats, room type their contact number to final a booking.
Roots	Roots allow clients to specify which directories servers should focus on, communicating intended scope through a coordination mechanism. Roots are **deprecated** as of protocol version `2026-07-28`.	A server for booking travel n be given access to a specifi directory, from which it can read a user's calendar.
Sampling	Sampling allows servers to request LLM completions through the client, enabling an agentic workflow. This approach puts the client in complete control of user permissions and security measures. Sampling is deprecated as of protocol version `2026-07-28`.	A server for booking travel n send a list of flights to an LL and request that the LLM pic the best flight for the user.

Roots:

- Defines boundaries and lists the accessible file-system paths or environmental scopes that the server is allowed to interact with.
- Roots define filesystem boundaries for server operations, allowing

clients to specify which directories servers should focus on.

- **Root structure :**
  - Roots are exclusively filesystem paths and always use the file:// URI scheme. They help servers understand project boundaries, workspace organisation, and accessible directories. The roots list can change as users work with different projects or folders. Servers pick up the updated boundaries the next time they request the roots list.

```
{
  "uri": "file:///Users/agent/travel-planning",
  "name": "Travel Planning Workspace"
}
```

The client provides filesystem roots to the travel planning server:

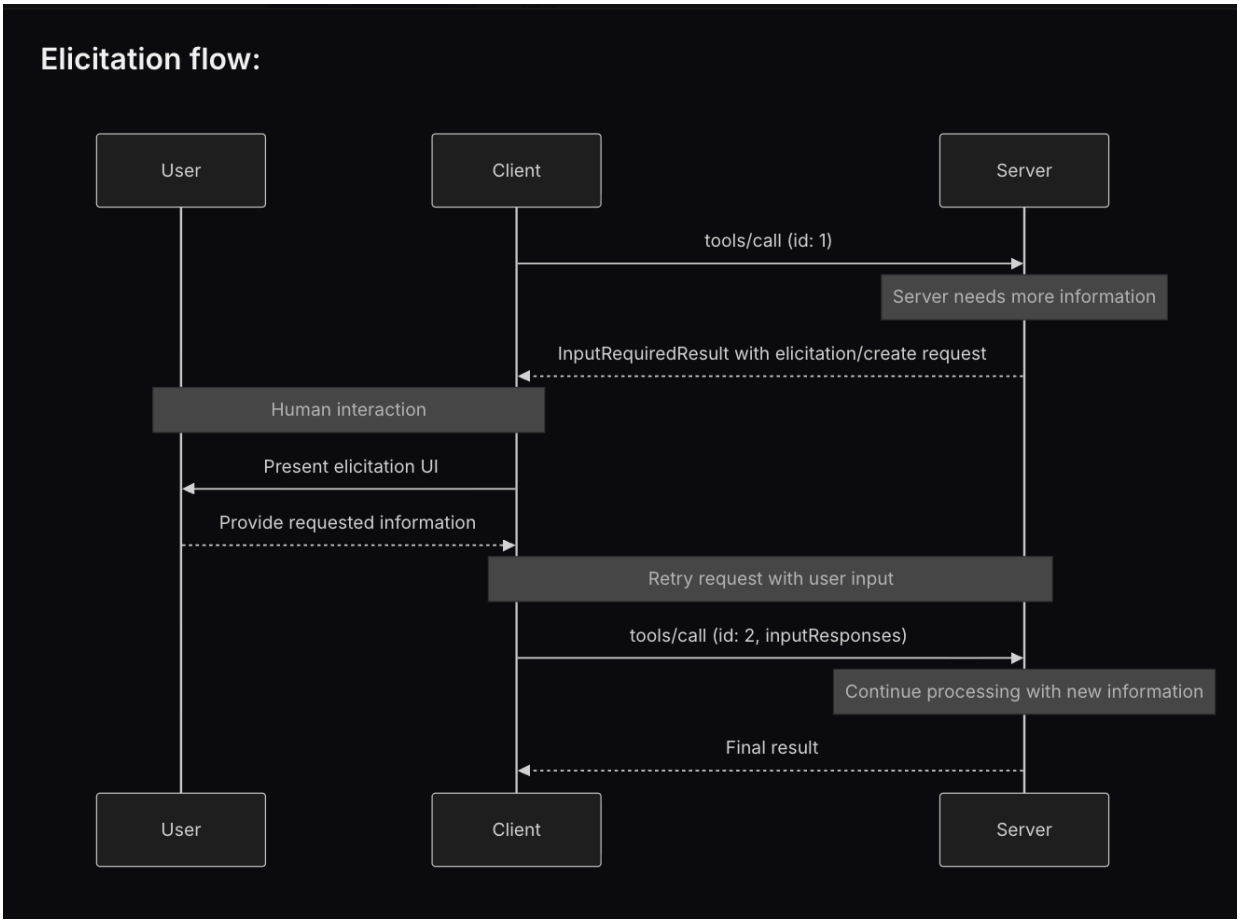
- file:///Users/agent/travel-planning - Main workspace containing all travel files
- file:///Users/agent/travel-templates - Reusable itinerary templates and resources
- file:///Users/agent/client-documents - Client passports and travel documents

Elicitation:

- Elicitation enables servers to request specific information from users during interactions, providing a structured way for servers to gather information on demand.
  - Allows a MCP Server to actively pause a long-running process and ask the MCP Client (and by extension, the user) for missing input or confirmation before proceeding.

- **Elicitation supports two modes:**
  - **Form mode**:
    - The server asks the client to collect structured data from the user. The request includes a schema that the client uses to build an input form and validate the response.
  - **URL mode**:
    - The server provides a URL for the user to open. The interaction happens out of band and its data never passes through the client, which makes this mode suitable for sensitive flows such as credential entry or third-party OAuth authorisation.
- Elicitation follows the **Multi Round-Trip Requests** (MRTR) pattern. When a server needs user input while processing a request such as tools/call, it responds with an InputRequiredResult whose inputRequests field carries one or more elicitation/create requests. The client gathers the input and retries the original request, attaching the collected inputResponses and echoing back any requestState the

server included.

- 

  - The flow enables dynamic information gathering. Servers can request specific data when needed, users provide information through appropriate UI, and servers complete the retried request with the newly acquired context.
  - **Elicitation request example (delivered inside** InputRequiredResult.inputRequests**):**

```
{
  method: "elicitation/create",
  params: {
    mode: "form",
    message: "Please confirm your United Kingdom vacation
booking details:",
    requestedSchema: {
      type: "object",
      properties: {
        confirmBooking: {
          type: "boolean",
          description: "Confirm the booking (Flights + Hotel
= $3,000)"
        },
        seatPreference: {
          type: "string",
          enum: ["window", "aisle", "no preference"],
```

```
          description: "Preferred seat type for flights"
        },
        roomType: {
          type: "string",
          enum: ["sea view", "city view", "garden view"],
          description: "Preferred room type at hotel"
        },
        travelInsurance: {
          type: "boolean",
          default: false,
          description: "Add travel insurance ($150)"
        }
      },
      required: ["confirmBooking"]
    }
  }
}
```

Sampling:

- Allows a MCP Server to ask the MCP Client's LLM to generate text or handle a sub-task mid-process. *(Note: As mentioned in the data layer documentation, this specific primitive is deprecated in newer versions of the protocol)*

Specification :

- MCP is an open protocol that enables seamless integration between LLM applications and external data sources and tools.
- Whether you're building an AI-powered IDE, enhancing a chat interface, or creating custom AI workflows, MCP provides a standardised way to connect LLMs with the context they need.

Over View :

- MCP provides a standardized way for applications to:
  - Share contextual information with language models
  - Expose tools and capabilities to AI systems
  - Build composable integrations and workflows
- The protocol uses **JSON-RPC** 2.0 messages to establish communication between:
  - **Hosts**: LLM applications that initiate connections
  - **Clients**: Connectors within the host application
  - **Servers**: Services that provide context and capabilities

Key Details :

Base Protocol

    - **JSON-RPC** message format
    - Stateless, self-contained requests
    - Per-request capability negotiation

Features

- (MCP Server Side Primitives) Servers offer any of the following features to clients:
    - **Resources**: Context and data, for the user or the AI model to use
    - **Prompts**: Templated messages and workflows for users
    - **Tools**: Functions for the AI model to execute

- (MCP Client Side Primitive) Clients may offer the following features to servers:
    - **Elicitation**: Server-initiated requests for additional information from users

Additional Utilities

- Configuration
- Progress tracking
- Cancellation
- Error reporting

Extensions

- Beyond the core protocol, MCP defines optional **extensions** that add modular, specialized, or experimental functionality. Extensions are always opt-in and require explicit support from both client and server, negotiated during initialization. Notable extensions include:
    - **Tasks**: Asynchronous execution of long-running operations, with polling, mid-flight input, and durable handles
    - **Skills over MCP**: Rich, structured instructions for agent workflows, discovered and consumed through MCP
    - **MCP Apps**: Interactive UI elements (charts, forms, video players) rendered inline within conversations

Security and Trust & Safety

- The Model Context Protocol enables powerful capabilities through arbitrary data access and code execution paths. With this power comes important security and trust considerations that all implementors must carefully address.

Key Principles

1. **User Consent and Control**
    - Users must explicitly consent to and understand all data access

and operations
    - Users must retain control over what data is shared and what actions are taken
    - Implementors should provide clear UIs for reviewing and authorizing activities
2. **Data Privacy**
    - Hosts must obtain explicit user consent before exposing user data to servers
    - Hosts must not transmit resource data elsewhere without user consent
    - User data should be protected with appropriate access controls
3. **Tool Safety**
    - Tools represent arbitrary code execution and must be treated with appropriate caution.
        - In particular, descriptions of tool behavior such as annotations should be considered untrusted, unless obtained from a trusted server.
    - Hosts must obtain explicit user consent before invoking any tool
    - Users should understand what each tool does before authorizing its use.

Question 1: Map your Day 7 agent. If tools.ts moved into MCP, which part of your code is the host, which is the client, and which is the server? Where does the model call happen

- MCP Server: Tools.ts is the core part of MCP Server.
- MCP Client: The connection object inside in the agent.ts.
- MCP Host: Create MCP Clients for every Server to establish connection. So agent.ts is MCP Host.
- Agent.ts (the host), which calls the Messages API. The MCP Server never calls the model.

Question 2: Tools, resources and prompts. Who decides when each one is used: the model, the application, or the user? Classify these three: get_exchange_rate, the list of a user's
saved conversations from Day 9, and a "summarize this conversation" template.

- The three primitives are independent. A user picks a prompt, the host expands it into messages, and the model sees only the result.
- get_exchange_rate : **The Model** decide to call this tool if it needs to execute currency conversion while answering you prompt.
- List of saved conversations : **The Application** decide (from your express server). It is data the application attaches as context.
- Summarize this conversation: **The User** — You choose this template to structure your instructions and focus the AI on what it needs to write

before it initiates its reasoning process.
- if you wanted the model to fetch the list on demand (“show my conversations”), it would be a tool instead. Which one do you want, and why?
- I also choose tool over a Resource. Because it aligns better with user intent and keeps your context window efficient.

Question 3: Transports. Your Week 3 plan includes an MCP server for UserAuth. Would you use stdio or Streamable HTTP, and why? For stdio, where must logs go? For HTTP, where does authentication happen, and how does that connect to your Day 9 JWT work?
Answer:

- For UserAuth I will use Streamable HTTP. A UserAuth system naturally implies a network-accessible service. By using Streamable HTTP. Backend can be hosted on remote cloud server, scale independently and cleanly stream real-time photo meta data notification or progress tracking.
- For STDIO logs must go strictly to standard error stderr. It keeps the communication channel pure.
- When using HTTP transport layer, authentication happen for each HTTP Request which carries Authorization: Bearer <token>.
- MCP Server verifies the signed JWT with payload , JWT Algorithm which HS256 and Access Secret Key withought using user auth backend. (Same like Day-9)
- the MCP server would hold the shared HS256 secret too, and the architecture page recommends OAuth for HTTP tokens.

Question 4: Statelessness. What does “stateless” mean for MCP, and how is it the same idea as the Messages API on Day 2? What do you lose, and what do you gain?
- MCP Statelessness means MCP Server has zero memory for past interactions. So every request to MCP Server By JSON-RPC is self contained. It provides **everything needed to process it**.
- What you lose: MCP Client needs to explicitly re-send in JSON-RPC formate everything every time, like version, capability information.
- What you gain: So Any Request Reach any Server Request , So horizontal scaling, Load Balancing and re-starts are simple.
- Because Inputs are Explicit and self contained, response becomes highly determined. This allows you to Cache Scope and Time to Live in Milliseconds.

Question 5: Cost. On Day 7 you measured about 957 fixed tokens for three tools plus the system prompt. What happens to that number when a host connects to five MCP servers with 30 tools? Look at “progressive

tool discovery" in the client best practices to see what the spec suggests.

- **Naive Total Preloading => 4,500 – 9,000 tokens => 22,500 – 45,000 tokens** *(Context Bloat - across multi turn)*
- Instead of overloading the prompt by dumping all 30 full JSON schemas at startup, the client implements a **layered, on-demand retrieval process**. First, it exposes only a compact "summary index" or a meta-search tool containing brief tool names and high-level category descriptions, which consumes minimal token overhead. When the model determines it needs a specific capability based on user input, it invokes a discovery query to retrieve that specific schema. Finally, the client injects only the necessary, fully-hydrated tool parameter blocks into the immediate context window for execution, discarding the other irrelevant tool specifications completely.